Correction complète — Programmation Impérative IV (TD2 à TD6)

Université Paris XIII — Christophe Tollu

TD2 — Arbres binaires et parsing d'expressions arithmétiques

Interface supposée (`binary_tree.h`)

```c
typedef char item;
typedef struct node *link;
struct node { item val; link left, right; };

link  cons_binary_tree(item v, link l, link r); // alloue un nœud
item  get_binary_tree_root(link h);             // renvoie h->val
int   less(item a, item b);                     // a < b
```

Exercice 1

a) Valeur d'un élément de EA (définition inductive)

Soit `e ∈ EA`. On définit `val(e)` par induction sur la structure de `e` :

- **Base :** si `e = d` avec `d ∈ {0,…,9}`, alors `val(e) = d` (valeur numérique du chiffre).
- **Cas inductif :**
  - si `e = +fg` avec `f, g ∈ EA`, alors `val(e) = val(f) + val(g)` ;
  - si `e = *fg` avec `f, g ∈ EA`, alors `val(e) = val(f) × val(g)`.

---

b) `parse_expr`

L'idée : lire le premier caractère. S'il est `+` ou `*`, parser récursivement deux sous-expressions. S'il est un chiffre, créer une feuille.

On passe un **pointeur sur le pointeur de lecture** pour avancer dans la chaîne :

```c
/* Fonction auxiliaire récursive; *s avance dans la chaîne */
static link parse_aux(const char **s) {
    char c = **s;
    (*s)++;                          /* avancer d'un caractère */
    if (c == '+' || c == '*') {
        link left  = parse_aux(s);
        link right = parse_aux(s);
        return cons_binary_tree(c, left, right);
    } else {
        /* c est un chiffre : feuille */
        return cons_binary_tree(c, NULL, NULL);
    }
}

/* Point d'entrée */
link parse_expr(const char *s) {
    return parse_aux(&s);
}
```

Terminaison : chaque appel récursif avance `*s` d'au moins un caractère ; la chaîne est finie donc la récursion termine.

---

c) `eval_tree`

```c
int eval_tree(link h) {
    char c = get_binary_tree_root(h);
    if (h->left == NULL)           /* feuille = chiffre */
        return c - '0';
    int l = eval_tree(h->left);
    int r = eval_tree(h->right);
    if (c == '+') return l + r;
    /* c == '*' */  return l * r;
}
```

---

d) `tree_to_expr` (écriture préfixe)

On construit la chaîne `c ++ left_str ++ right_str` pour un nœud interne, et `"c"` pour une feuille.

```c
#include <stdlib.h>
#include <string.h>

char *tree_to_expr(link h) {
    char c = get_binary_tree_root(h);
    if (h->left == NULL) {          /* feuille */
        char *s = malloc(2);
        s[0] = c;  s[1] = '\0';
        return s;
    }
    char *left  = tree_to_expr(h->left);
    char *right = tree_to_expr(h->right);
    /* longueur : 1 (opérateur) + |left| + |right| + 1 (terminateur) */
    size_t len = 1 + strlen(left) + strlen(right) + 1;
    char *s = malloc(len);
    s[0] = c;  s[1] = '\0';
    strcat(s, left);
    strcat(s, right);
    free(left);
    free(right);
    return s;
}
```

TD3 — Parsing (fin) et tri par tas

Exercice 1 — `tree_to_infix`

On produit `"(" ++ left_infix ++ c ++ right_infix ++ ")"` pour chaque nœud interne :

```c
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

char *tree_to_infix(link h) {
    char c = get_binary_tree_root(h);
    if (h->left == NULL) {          /* feuille */
        char *s = malloc(2);
        s[0] = c;  s[1] = '\0';
        return s;
    }
    char *left  = tree_to_infix(h->left);
    char *right = tree_to_infix(h->right);
    /* format : ( left c right ) */
    size_t len = 1 + strlen(left) + 1 + strlen(right) + 1 + 1;
    char *s = malloc(len);
    sprintf(s, "(%s%c%s)", left, c, right);
    free(left);
    free(right);
    return s;
}
```

Exemple : l'arbre de `+*35*+426` donne `((3*5)+((4+2)*6))`.

Exercice 2 — `heap_sort`

On trie le segment `t[l..r]` ($n = r-l+1$ éléments) **en place**, sans mémoire annexe.

Pour un **tas-max** stocké dans `t[l..r]`, le nœud d'indice absolu `i` a :

- enfant gauche : `2*(i-l)+1 + l`
- enfant droit : `2*(i-l)+2 + l`
- parent : `l + (i-l-1)/2`

Fonction auxiliaire : `sift_down`

Fait « descendre » l'élément `t[i]` pour restaurer la propriété de tas-max sur `t[l..end]`.

```c
static void swap(int *t, int i, int j) {
    int tmp = t[i]; t[i] = t[j]; t[j] = tmp;
}

/* Fait descendre t[i] dans le tas t[l..end] (tas-max) */
static void sift_down(int *t, int l, int end, int i) {
    while (1) {
        int left  = l + 2*(i-l) + 1;
        int right = l + 2*(i-l) + 2;
        int max   = i;
        if (left  <= end && t[left]  > t[max]) max = left;
        if (right <= end && t[right] > t[max]) max = right;
        if (max == i) break;
        swap(t, i, max);
        i = max;
    }
}
```

`heap_sort`

```c
void heap_sort(int *t, int l, int r) {
    int n = r - l + 1;

    /* Étape 1 : construction du tas-max par « heapify » ascendant.
       Complexité : O(n) — mais O(n log n) est aussi acceptable. */
    for (int i = l + (n-2)/2; i >= l; i--)
        sift_down(t, l, r, i);

    /* Étape 2 : tri — extraire successivement le maximum.
       Complexité : O(n log n). */
    for (int end = r; end > l; end--) {
        swap(t, l, end);            /* le max (racine) va en fin */
        sift_down(t, l, end-1, l); /* restaurer le tas sur t[l..end-1] */
    }
}
```

Pourquoi le tas-max ?
Pour trier dans l'ordre **croissant**, on place le plus grand en dernière position à chaque étape. Un tas-max donne accès immédiat au plus grand élément (la racine).

Complexité :

- Étape 1 (heapify ascendant) : O(n) — voire O(n log n) par insertions successives.
- Étape 2 : n−1 appels à `sift_down`, chacun en O(log n) → O(n log n).
- **Total : O(n log n), en place, sans mémoire annexe.** ✓

TD4 — Arbres binaires de recherche (ABR)

On suppose que les nœuds ont un champ `size` mis à jour par les rotations, et que `size(h)` renvoie 0 si `h == NULL`.

Exercice 1 — `insert_root_BST_it`

Idée : descendre dans l'arbre en empilant, à chaque étape, un **pointeur vers le lien** à mettre à jour et la direction empruntée. En remontant, appliquer les rotations inverses.

c

```c
#include <stdlib.h>  /* pour la pile sur le tas si nécessaire */

#define MAX_H 64     /* hauteur maximale raisonnable */

link insert_root_BST_it(link h, item v) {
    link  *slots[MAX_H]; /* slots[i] = adresse du lien de niveau i */
    int    dirs [MAX_H]; /* 0 = allé à gauche, 1 = allé à droite */
    int    top = 0;

    link *slot = &h;     /* curseur sur le lien courant */

    /* Descente : trouver la position d'insertion */
    while (*slot != NULL) {
        link cur = *slot;
        if (less(v, get_binary_tree_root(cur))) {
            slots[top] = slot;
            dirs [top] = 0;
            top++;
            slot = &(cur->left);
        } else {
            slots[top] = slot;
            dirs [top] = 1;
            top++;
            slot = &(cur->right);
        }
    }

    /* Créer le nouveau nœud à la feuille */
    *slot = cons_binary_tree(v, NULL, NULL);

    /* Remontée : appliquer les rotations (ordre inverse) */
    while (top > 0) {
        top--;
```

```
        if (dirs[top] == 0)
            *slots[top] = rotate_right(*slots[top]);
        else
            *slots[top] = rotate_left (*slots[top]);
    }

    return h;
}
```

Correction : à chaque niveau on effectue la même rotation que dans la version récursive — la dérécursification est fidèle. ✓

Exercice 2a — `partition_BST`

Objectif : faire remonter à la racine le nœud de **rang k** (0-indexé, rang = nombre d'éléments strictement inférieurs).

```c
int size(link h) { return h == NULL ? 0 : h->size; }
/* On suppose que size est mis à jour dans cons_binary_tree et les rotations. *

link partition_BST(link h, int k) {
    if (h == NULL) return NULL;
    int m = size(h->left);   /* rang de la racine actuelle */
    if (k < m) {
        h->left = partition_BST(h->left, k);
        h = rotate_right(h);
    } else if (k > m) {
        h->right = partition_BST(h->right, k - m - 1);
        h = rotate_left(h);
    }
    /* si k == m : la racine est déjà de rang k */
    return h;
}
```

L'arbre renvoyé est un ABR : les rotations préservent la propriété d'ordre des ABR (elles ne font que remonter un nœud sans changer les relations d'ordre entre étiquettes). ✓

Propriété : après l'appel, le sous-arbre gauche de la racine a exactement `k` nœuds. ✓

Exercice 2b — `balance_BST`

On rééquilibre récursivement en amenant la médiane à la racine.

Pour `n` nœuds, le rang médian est `k = (n-1)/2` (division entière) — le sous-arbre gauche aura $\lfloor (n-1)/2 \rfloor$ nœuds et le droit $\lceil (n-1)/2 \rceil$ nœuds, soit une différence d'au plus 1.

```c
link balance_BST(link h) {
    if (h == NULL) return NULL;
    int n = size(h);
    if (n <= 1)  return h;
    int k = (n - 1) / 2;
    h         = partition_BST(h, k);   /* médiane à la racine */
    h->left   = balance_BST(h->left);
    h->right  = balance_BST(h->right);
    return h;
}
```

L'arbre renvoyé est un ABR : `partition_BST` préserve la propriété ABR ; les appels récursifs la préservent par induction. ✓

L'arbre est équilibré : par induction, les sous-arbres sont équilibrés avec des tailles $\lfloor (n-1)/2 \rfloor$ et $\lceil (n-1)/2 \rceil$ dont les hauteurs diffèrent d'au plus 1. ✓

Exercice 3 — `delete_node_BST`

On utilise `partition_BST` pour fusionner le sous-arbre gauche `L` et le sous-arbre droit `R` après suppression de la racine : on amène le **minimum de R** (rang 0) à la racine de R, puis on attache L comme sous-arbre gauche.

```c
link delete_node_BST(link h, item v) {
    if (h == NULL) return NULL;
    item root_val = get_binary_tree_root(h);
    if (less(v, root_val)) {
        h->left  = delete_node_BST(h->left,  v);
    } else if (less(root_val, v)) {
        h->right = delete_node_BST(h->right, v);
    } else {
        /* Trouvé : supprimer h */
        link L = h->left, R = h->right;
        free(h);                        /* libérer le nœud */
        if (R == NULL) return L;
        if (L == NULL) return R;
        /* Amener le minimum de R (rang 0) à la racine de R */
        R = partition_BST(R, 0);
        R->left = L;                    /* attacher L */
        return R;
    }
    return h;
}
```

L'arbre renvoyé est un ABR :

- Le minimum de R est supérieur à toutes les étiquettes de L (car v les sépare et v est supprimé).
- Attacher L à gauche du minimum de R préserve donc l'ordre. ✓

TD5 — ABR, arbres 2-3 et LLRBT

Exercice 1a — Preuve que k'' est un ABR

Soient `h` et `k` deux ABR. On insère la racine de `h` (valeur `v`) à la racine de `k` → `k'`. La racine de `k'` est `v`, son sous-arbre gauche `k'_L` contient toutes les étiquettes de `k` inférieures à `v`, et son sous-arbre droit `k'_R` contient celles supérieures à `v`.

On insère ensuite la racine de `h_L` (valeur `v_L < v`) dans `k'_L` → `k''_L`, et la racine de `h_R` (valeur `v_R > v`) dans `k'_R` → `k''_R`.

`k''` est l'arbre de racine `v`, de sous-arbre gauche `k''_L` et droit `k''_R`.

- Toutes les étiquettes de `k''_L` sont `< v` (car elles viennent de `k'_L` + `v_L`, tous `< v`).
- Toutes les étiquettes de `k''_R` sont `> v`.
- `k''_L` et `k''_R` sont des ABR par hypothèse d'induction.
- Donc `k''` est un ABR. ✓

Exercice 1b — `join_BST`

```c
link join_BST(link h, link k) {
    if (h == NULL) return k;
    if (k == NULL) {
        /* On doit insérer toutes les étiquettes de h dans un arbre vide.
           Pour ne pas modifier la mémoire nette : allouer autant que libérer.
           On crée les nœuds dans k et libère ceux de h au fur et à mesure. */
        item v      = get_binary_tree_root(h);
        link h_l    = h->left, h_r = h->right;
        free(h);
        link result = cons_binary_tree(v, NULL, NULL);
        result      = join_BST(h_l, result);
        result      = join_BST(h_r, result);
        return result;
    }
    /* Insérer la racine de h à la racine de k */
    item v   = get_binary_tree_root(h);
    link h_l = h->left, h_r = h->right;
    free(h);                            /* libérer le nœud de h */
    k = insert_Root_BST(k, v);         /* alloue un nouveau nœud → bilan nul */
    /* Fusionner les sous-arbres */
    k->left  = join_BST(h_l, k->left);
    k->right = join_BST(h_r, k->right);
    return k;
}
```

Bilan mémoire nul : à chaque appel, on libère un nœud de `h` (`free(h)`) et `insert_Root_BST` en alloue un nouveau. Le nombre total de nœuds alloués est donc conservé. ✓

Exercice 1c — Complexité de `join_BST`

Soient `n = |h|` et `m = |k|`.

À chaque appel récursif : `insert_Root_BST(k, v)` coûte O(hauteur de k) = **O(m)** dans le pire cas (ABR dégénéré = chaîne).

La récurrence sur la structure de `h` donne (pire cas : `h` est une chaîne et `k` idem) :

```
T(n, m) = O(m) + T(n-1, m-1)
        = O(m) + O(m-1) + … + O(1)
        = Θ(nm)
```

Donc `join_BST(h, k)` effectue **Θ(nm)** comparaisons et affectations dans le pire cas. ✓

Exercice 2a — Vérification que l'arbre est un LLRBT

L'arbre initial (liens rouges : J→I, S→Q, W→U) :

```
          M
        /   \
      G       S
     / \     / \
   D   J   Q     W
       /   /\    /
      I   O  R  U
```

Vérifications des propriétés LLRBT :

1. **Liens rouges tous à gauche :** I est enfant gauche de J ✓, Q est enfant gauche de S ✓, U est enfant gauche de W ✓.
2. **Pas deux liens rouges consécutifs :** aucun parent de I, Q, U n'est lui-même rouge ✓.
3. **Hauteur noire uniforme** (chaque chemin racine → feuille a le même nombre de liens noirs) :
   - M→G→D→∅ : 3 liens noirs ✓
   - M→G→J→I→∅ : M-G(N) + G-J(N) + J-I(R) + I-∅(N) = 3 noirs ✓
   - M→S→Q→O→∅ : M-S(N) + S-Q(R) + Q-O(N) + O-∅(N) = 3 noirs ✓
   - M→S→W→U→∅ : M-S(N) + S-W(N) + W-U(R) + U-∅(N) = 3 noirs ✓
   - (et R, et tous les ∅ de feuilles) ✓

Conclusion : c'est bien un LLRBT. ✓

Exercice 2b — Arbre 2-3 correspondant

Dans un LLRBT, un lien rouge fusionne un nœud avec son parent pour former un **3-nœud**. On regroupe donc :

- J et I → 3-nœud **[I, J]**
- S et Q → 3-nœud **[Q, S]**
- W et U → 3-nœud **[U, W]**

Arbre 2-3 résultant :

```
              [M]
            /     \
         [G]      [Q, S]
        /   \    /  |   \
     [D]  [I,J] [O] [R] [U,W]
```

Exercice 2c — Insertions dans le LLRBT

Méthode générale :

1. Insérer en feuille (lien rouge).
2. Remonter en appliquant les corrections :
   - Lien rouge à droite → `rotate_left`
   - Deux liens rouges consécutifs à gauche → `rotate_right`
   - Deux enfants rouges → `color_flip` (puis propager la couleur rouge vers le parent)
3. La racine est toujours noire.

Résumé des insertions (sans détailler tous les pas) :

Lettre	Position	Opérations nécessaires
A	feuille gauche de D	rotate_right(G), color_flip(M)
B	entre A et D	plusieurs rotations + flip
C	idem	idem
E	feuille droite de D	rotate_left(D), rotate_right(G)
F	entre E et G	rotations + flip
H	feuille droite de G (entre G et I)	rotate_left(G) puis remontée
K	entre J et M	rotate_left(J)
L	entre K et M	rotations
N	entre M et O	color_flip si nécessaire
P	entre O et Q	rotations
T	entre S et U	rotate_left(S) puis remontée
V	entre U et W	rotate_left(U)
X	feuille droite de W	remontée simple
Y	après X	rotations + flip
Z	après Y (ou directement à droite de W)	**voir détail ci-dessous**

Exercice 2d — LLRBT après insertion de Z

Simulation détaillée de l'insertion de Z :

Z est le plus grand élément → il descend jusqu'à devenir enfant droit de W (nœud rouge).

Étape par étape (en remontant) :

1. **Au nœud W :** W.gauche = U (rouge), W.droite = Z (rouge, nouveau).
   - Deux enfants rouges → **color_flip(W)** : W devient rouge, U et Z deviennent noirs.
2. **Au nœud S :** S.gauche = Q (rouge), S.droite = W (maintenant rouge).
   - Deux enfants rouges → **color_flip(S)** : S devient rouge, Q et W deviennent noirs.
3. **Au nœud M :** M.gauche = G (noir), M.droite = S (maintenant rouge).
   - Lien rouge à droite → **rotate_left(M)** :
     - S devient la nouvelle racine (noire).
     - M devient enfant gauche de S (rouge).
     - Q (maintenant noir) devient enfant droit de M.
4. **Résultat final :** S est la racine. On vérifie : S.gauche = M (rouge), S.droite = W (noir). Pas d'autres violations.

LLRBT après insertion de Z :

```
            S   ← nouvelle racine (noire)
          /   \
        M       W        (S→M : rouge ; S→W : noir)
      /   \    / \
     G     Q  U   Z      (tous liens noirs)
    / \   / \
   D   J O   R
      /
     I                  (J→I : rouge)
```

Liens rouges dans l'arbre final : S→M et J→I.

(Q, anciennement enfant rouge de S, est devenu enfant droit de M avec un lien noir suite à la rotation.)

TD6 — Arbres équilibrés et tris

Exercice 1a — `tri_RB`

On insère tous les éléments dans un LLRBT, puis on les récupère par **parcours infixe** (qui donne l'ordre croissant pour un ABR).

```c
/* Parcours infixe : remplit tab à partir de l'indice *i */
static void inorder(link h, int *tab, int *i) {
    if (h == NULL) return;
    inorder(h->left,  tab, i);
    tab[(*i)++] = (int)(get_binary_tree_root(h) - '0'); /* adapter selon item *
    inorder(h->right, tab, i);
}

static void free_tree(link h) {
    if (h == NULL) return;
    free_tree(h->left);
    free_tree(h->right);
    free(h);
}

void tri_RB(int *tab, int n) {
    link tree = NULL;
    /* Insertion dans le LLRBT : O(log n) par insertion */
    for (int i = 0; i < n; i++)
        tree = insert_LLRBT(tree, tab[i]);   /* insert de Sedgewick */
    /* Récupération en ordre croissant : O(n) */
    int idx = 0;
    inorder(tree, tab, &idx);
    free_tree(tree);
}
```

Complexité :

- n insertions dans un LLRBT → O(log n) chacune (hauteur bornée par 2 log n) → **O(n log n)** au total.
- Parcours infixe : **O(n)**.
- **Coût total : O(n log n).** ✓

Exercice 1b — Avec un ABR « standard »

Dans le pire des cas, si le tableau est déjà trié (croissant ou décroissant), l'ABR se dégénère en une liste chaînée de hauteur n.

- La k-ième insertion coûte O(k) → coût total des insertions : O(1) + O(2) + … + O(n) = **$O(n^2)$**.
- Le parcours infixe reste O(n).

Fonction de coût dans le pire des cas : $O(n^2)$.

(Par exemple, trier un tableau déjà trié avec un ABR standard donne systématiquement le pire cas.)

Fin de la correction — Prog4 TD2 → TD6